Unidade 6 — Ferramentas, MCP e Skills

Aplicações de Modelos de Linguagem Natural

O que são ferramentas

- Sozinhos, os LLMs são funções que recebem texto e devolvem texto. **Ferramentas** (*tools*), na forma de funções e APIs, permitem que eles interajam com qualquer sistema exposto a eles, de bases de código e ferramentas internas até bancos de dados online e assistentes domésticos.
- Um traço que define um agente é a capacidade de **procurar, escolher e usar ferramentas de forma autônoma**; agentes muito capazes podem até criar as suas próprias ferramentas.
- Ferramentas também servem para acessar memórias e conhecimento externo (como no **RAG agêntico**) ou até outros LLMs especializados (multimodais ou de programação).
- Sem ferramentas, um agente pode pensar e planejar, mas **não pode agir**.
- O grau de autonomia define como as ferramentas são usadas:
  - Em um **fluxo fixo**, as ferramentas são chamadas em uma ordem predefinida: por exemplo, um agente de pesquisa sempre consulta o arXiv e o Google e depois resume.
  - Com mais autonomia, o **agente escolhe** qual ferramenta usar e quando: por exemplo, busca primeiro no Google, depois no arXiv, e então responde.

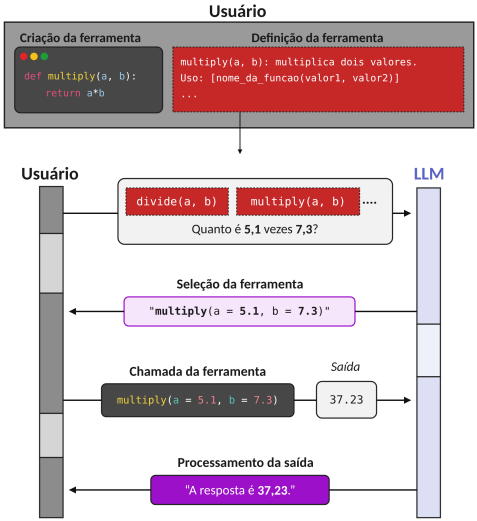
As etapas do uso de ferramentas

Usar uma ferramenta envolve mais passos do que parece. O processo pode ser dividido em cinco etapas:

1. **Criação**;
2. **Definição**;
3. **Seleção**;
4. **Chamada**;
5. **Processamento da saída**.

Lembrete

O LLM não chama a ferramenta; ele **mostra a intenção** de chamá-la, e a execução é feita por um processo externo (automatizado).



Etapa 1: criação

- Ferramentas costumam ser funções acessadas por uma API ou por código interno.
- Como os LLMs não são conhecidos pelas habilidades matemáticas, o exemplo é uma função `multiply(a, b)` que multiplica dois valores (recebidos como **texto**, porque o LLM só produz texto).
- As ferramentas devem ser **bem documentadas**, como se fossem feitas para pessoas:
  - o usuário entende o que a ferramenta faz?
  - o retorno está claro?
  - os nomes das variáveis são descritivos?
- Isso é ainda mais importante para LLMs, que precisam ser informados explicitamente do que existe e do que cada ferramenta consegue fazer.

Etapa 2: definição (treinamento ou prompt)

Depois de criada, é preciso informar ao LLM o que a ferramenta faz, como é usada e em que contexto. Há dois caminhos:

- O uso de ferramentas pode ser **aprendido no treinamento**: modelos recentes, como **Qwen3** e **DeepSeek-V3.2**, aprendem a reconhecer definições de ferramentas no prompt e a usá-las de forma geral.
- Ou a definição pode ser passada **pelo prompt**, de duas formas. A primeira é descrever as ferramentas **em linguagem natural**, no prompt de sistema:

Definição em linguagem natural

```
# Definição das ferramentas
Você pode usar as seguintes ferramentas:
- multiply(a, b): multiplica dois números
- divide(a, b): divide a por b

# Uso das ferramentas
Quando precisar usá-las, escreva: [nome_da_funcao(valor1, valor2)]
```

Etapa 2: definição por chamada de função estruturada

A segunda forma é a **chamada de função estruturada** (*structured function calling*): cada ferramenta é descrita por um **esquema JSON** (*JSON Schema*) com nome, descrição e parâmetros (tipos e campos obrigatórios). As chaves do esquema fazem parte do formato e ficam em inglês.

Esquema JSON da ferramenta `multiply`

```
{
  "type": "function",
  "function": {
    "name": "multiply",
    "description": "Multiplica dois números",
    "parameters": {
      "type": "object",
      "properties": {
        "a": {"type": "number", "description": "Primeiro número"},
        "b": {"type": "number", "description": "Segundo número"}
      },
      "required": ["a", "b"]
    }
  }
}
```

Boas práticas de definição e etapa 3: seleção

Boas práticas de definição de ferramentas

- **Documente a ferramenta** com descrições completas.
- **Minimize o número de ferramentas** (menos de 10 é um bom começo).
- **Minimize o escopo de cada ferramenta**: funções com dezenas de parâmetros são difíceis de usar, principalmente para modelos menores.

Seleção

- O LLM precisa escolher a ferramenta certa (se alguma for necessária) e usá-la corretamente. Aqui os **LLMs de raciocínio** se destacam, pois podem "pensar" sobre qual ferramenta usar.
- Perguntas simples ("Quanto é $1 + 1$?") não exigem muito, mas planejar uma viagem com orçamento exige planejamento em vários passos.
- Quando há muitas ferramentas, encher a janela de contexto com todos os esquemas piora o desempenho, como vimos na engenharia de contexto. Uma solução é guardar os esquemas em um banco vetorial e **usar RAG para descobrir as ferramentas mais relevantes** para a pergunta.

Etapa 4: chamada

Chamada

- Ao terminar de raciocinar, o LLM produz uma **string** com a chamada formatada, por exemplo em JSON. Essa saída é apenas a intenção: um programa externo precisa extrair e interpretar o JSON e, então, executar a função.
- Nesse ponto, pode-se pedir **aprovação humana** antes de executar ferramentas perigosas (o humano no circuito da Seção 2.6).
- Bibliotecas como **jsonschema** e **Pydantic** ajudam a validar essas chamadas de forma mais robusta do que expressões regulares.

Etapa 5: processamento da saída

- O resultado da ferramenta precisa voltar ao LLM, com a mesma estrutura de mensagens da memória de conversa: acrescenta-se uma mensagem do **assistente** com a chamada e uma mensagem com o **resultado**.
- Em LLMs treinados para chamar ferramentas nativamente, existe um papel próprio, `tool`; nos demais, usa-se o papel `user` com o prefixo **OBSERVATION**, como se o usuário tivesse observado o resultado e o contasse ao LLM. Com essas mensagens, o LLM pode combinar, resumir ou apresentar melhor a resposta final.

Histórico de mensagens de uma chamada de ferramenta

```
system:    Você é um assistente prestativo. (+ definição das ferramentas)
user:      Quanto é 5,1 vezes 7,3?
assistant: {"tool": "multiply", "kwargs": {"a": "5.1", "b": "7.3"}}
user:      OBSERVATION: 37.23
assistant: A resposta é 37,23.
```

Ligação com a Unidade 2: THOUGHT (o raciocínio sobre o que fazer), **ACTION** (a chamada de ferramenta em JSON), **OBSERVATION** (o resultado da ação) e **ANSWER** (a resposta final, após um ou mais passos) são as etapas do framework **ReAct** (*Reason and Act*), que dá autonomia ao agente.

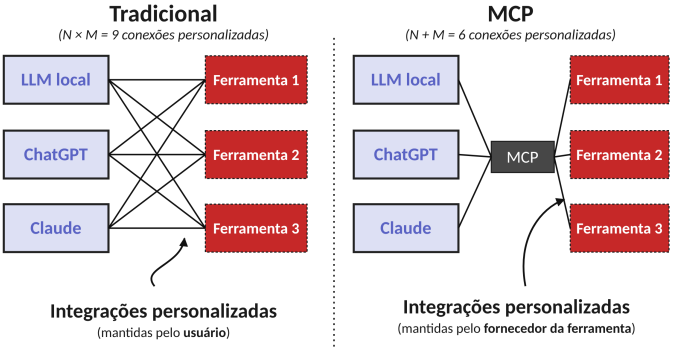
MCP: o problema N $\times$ M

- Imagine que você criou prompts e integrações para várias ferramentas e surge um novo LLM com uma forma diferente de chamar ferramentas: seria preciso refazer todas as integrações.
- Esse é o **problema N $\times$ M**: com N LLMs e M ferramentas, seria preciso escrever uma integração para cada combinação.
- O **Model Context Protocol (MCP)** resolve isso padronizando a forma de conectar ferramentas e APIs aos LLMs. É um **padrão aberto**, criado pela Anthropic e lançado em novembro de 2024, frequentemente chamado de "**porta USB-C da IA**".
- Com ele, o número de conexões a manter passa a ser **N + M**, e a manutenção da integração passa do usuário para o **fornecedor da ferramenta**.

Exemplo

Se a API do arXiv mudar, basta que os mantenedores atualizem o servidor MCP, e todos os usuários recebem a atualização.

MCP: de N × M para N + M



Sem o MCP, cada ferramenta precisaria ser:

- **rastreada** e passada ao LLM manualmente;
- **descrita** manualmente (incluindo o esquema JSON);
- **atualizada** manualmente sempre que a API mudasse.

Já existem servidores MCP para **Figma**, **GitHub**, **Home Assistant** e muitos outros serviços.

Skills: por que e o que são

- Ferramentas e MCP dão ao agente ações que ele pode executar, mas não dizem **quando** usá-las nem como elas se encaixam em um fluxo de trabalho maior.
- O agente não conhece o fluxo da sua equipe, as ferramentas preferidas ou os padrões de código; ter de repetir, a cada sessão, que ele deve usar `uv` em vez de `pip` é incômodo.
- As **Skills**, criadas pela Anthropic, são um conjunto de **instruções, scripts e recursos** que o agente usa como contexto para realizar uma tarefa.
- Elas contêm **conhecimento procedural**: instruções passo a passo de como abordar uma tarefa no seu contexto.

Exemplo

Por exemplo, para criar uma apresentação, o agente pode buscar informação na web (ferramenta), resumir (ele mesmo faz) e criar os slides um a um (ferramenta); a Skill é a "receita" que diz **o que fazer, quando e como**.

4.4 Skills: divulgação progressiva e o SKILL.md

Uma Skill funciona por **divulgação progressiva** (*progressive disclosure*): o agente começa com o mínimo de informação e recebe mais detalhes só quando a Skill é usada, em três camadas:

As três camadas

Camada 1 → metadados do SKILL.md (YAML: nome e descrição) → sempre carregada → cerca de 100 tokens

Camada 2 → instruções do SKILL.md (Markdown) → carregada quando a Skill é ativada → menos de 5.000 tokens

Camada 3 → arquivos anexos (Markdown, scripts, dados etc.) → carregados quando necessário → conforme a necessidade

- Usaremos como exemplo uma Skill que transforma transcrições e anotações de reuniões em atas estruturadas com itens de ação. Toda Skill precisa de um arquivo **SKILL.md**.
- O início do arquivo traz os **metadados em YAML**: o nome (`meeting_notes`) e a descrição ("Use esta Skill para transformar transcrições ou anotações de reuniões em decisões e itens de ação estruturados"). Os metadados de todas as Skills ficam sempre visíveis ao agente, geralmente no prompt de sistema, para que ele decida se vale a pena explorá-las.

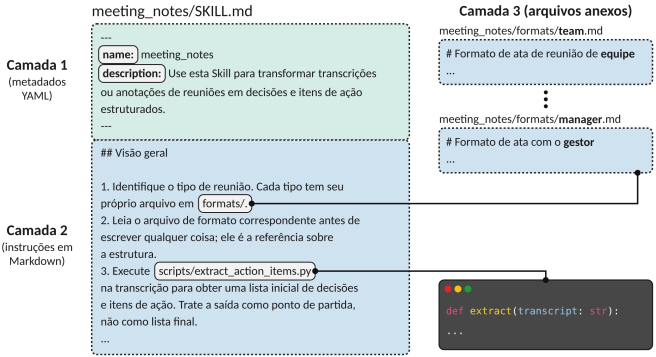
Skills: o exemplo `meeting_notes`

- O **corpo** do SKILL.md traz as instruções, entregues só quando o agente as pede, como se fosse uma ferramenta que fornece contexto: uma aplicação direta da engenharia de contexto.
- Quando a Skill cresce (atas de reunião individual com o gestor, com colegas, com a equipe, com parceiros externos), é possível **anexar arquivos** na pasta da Skill (outros arquivos Markdown e scripts Python) e referenciá-los no SKILL.md, para que o agente os carregue apenas quando necessário.

Estrutura de pastas

```
meeting_notes/
├── SKILL.md
├── scripts/
│   └── extract_action_items.py
└── formats/
    ├── one_on_one_manager.md
    ├── one_on_one_colleague.md
    ├── one_on_one_report.md
    ├── team_meeting.md
    └── external_call.md
```

Skills: anatomia de uma Skill



Em resumo

Enquanto a definição de uma ferramenta diz ao agente **o que** ela faz, uma Skill diz **como** realizar um trabalho (os passos, as convenções e as referências de uma tarefa recorrente), mantendo a janela de contexto enxuta.